

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: B.Charishma

Roll No.: a24126552067

Date: 30-08-2026

1. TITLE

Interactive DOM Manipulation and Object-Oriented Architecture

2. AIM

To architect an interactive web dashboard using JavaScript Event Listeners, Prompt interactions, dynamic CSS modifications, and ES6 Object-Oriented Class hierarchies.

3. OBJECTIVE

The objectives of this experiment are:

- To attach dynamic event listeners to DOM buttons.
- To modify CSS styling directly via the JavaScript Style Object.
- To capture user input synchronously via browser Prompts.
- To model data using ES6 classes and inheritance structures.
- To execute class methods that update the User Interface dynamically.

4. THEORY

DOM Manipulation is the process of interacting with the DOM API to change or modify the HTML document dynamically. Event listeners observe specific elements and trigger callback functions when user events (clicks, keypresses) occur.

JavaScript Classes allow developers to construct blueprints for data models. Inheritance (using extends) enables specialized subclasses (like AdvancedCourse) to inherit properties from a base class (like Course), keeping logic modular and DRY (Don't Repeat Yourself).

Application Flow:

User Event (Click) -> Event Listener Triggered -> Callback Execution -> Class Method Invocation -> DOM

Update

5. ALGORITHM / PROCEDURE

1. Build an HTML interface with several interaction buttons.
2. Define a base Course class with title, description, and DOM element ID properties.
3. Create an AdvancedCourse subclass that extends Course and adds a level property.
4. Implement a `showInfo()` method within the classes to inject data into the DOM.
5. Instantiate objects representing AI, Editing, and Game Development courses.
6. Use `addEventListener()` to bind click events on buttons to the objects' `showInfo()` methods.
7. Implement a separate event listener that triggers a `prompt()` dialog to capture user names.
8. Implement a function to dynamically alter `document.body.style.background`.
9. Verify all interactions and state changes within the browser.

6. PROGRAM / SOURCE CODE

Source File: `index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Dynamic Website Using JavaScript</title>

<style>
*{
    margin:0;
    padding:0;
    box-sizing:border-box;
}

body{
    font-family:Arial, sans-serif;
    background:linear-gradient(135deg,#74ebd5,#ACB6E5);
}

/* Header */
header{
    background:#333;
    color:white;
    text-align:center;
    padding:20px;
    text-shadow:2px 2px 5px black;
}

/* Navigation */
nav{
    background:#444;
    padding:15px;
    text-align:center;
}

nav a{
    color:white;
    text-decoration:none;
    margin:15px;
    font-weight:bold;
}

nav a:hover{
    color:yellow;
}

.des{
    text-align:center;
    padding:50px;
}
```

```
.des h2{
  margin-bottom:20px;
}

button{
  padding:10px 20px;
  margin:10px;
  border:none;
  border-radius:8px;
  background:rgb(190,120,250);
  color:white;
  cursor:pointer;
  font-size:16px;
}

button:hover{
  background:rgb(130,210,250);
}

/* Cards */
.container{
  display:flex;
  justify-content:center;
  gap:20px;
  padding:30px;
}

.card{
  width:250px;
  background:white;
  padding:20px;
  border-radius:10px;
  box-shadow:0 0 10px gray;
  text-align:center;
}

.card:hover{
  transform:scale(1.05);
}

/* Footer */
footer{
  background:#333;
  color:white;
  text-align:center;
  padding:15px;
  margin-top:30px;
}

</style>
</head>
```

```
<body>

<header>
  <h1>MY Dynamic Website</h1>
</header>

<nav>
  <a href="#">Home</a>
  <a href="#">About</a>
  <a href="#">Courses</a>
  <a href="#">Contact</a>
</nav>

<section class="des">

  <h2 id="message">Welcome to My Website</h2>

  <button id="msgBtn">Change Message</button>

  <button id="colorBtn">Change Background</button>

</section>

<div class="container">

  <div class="card">

    <h3>AI</h3>

    <p id="ai">
      AI is useful for the next generation.
    </p>

    <button id="btnAI">
      KNOW MORE
    </button>

  </div>

  <div class="card">

    <h3>EDITING</h3>

    <p id="edit">
      Editing helps you earn money.
    </p>

    <button id="btnEdit">
      KNOW MORE
    </button>

  </div>

</div>
```

```

<div class="card">

    <h3>GAME DEVELOPMENT</h3>

    <p id="game">
        Game Development is a growing career.
    </p>

    <button id="btnGame">
        KNOW MORE
    </button>

</div>

</div>

<footer>

    <p>&copy; A.ROHIT - Dynamic JavaScript Website</p>

</footer>

<script>

document.getElementById("msgBtn").addEventListener("click",changeMessage);

document.getElementById("colorBtn").addEventListener("click",changeColor);


const ai=new AdvancedCourse(
"AI",
"Artificial Intelligence enables machines to learn and solve problems.",
"ai",
"Advanced"
);

const editing=new Course(
"Editing",
"Video editing is useful for YouTube, Films and Social Media.",
"edit"
);

const game=new AdvancedCourse(
"Game Development",
"Game Development uses HTML, CSS, JavaScript, Unity and C#.",
"game",

```

```
"Intermediate"
);

document.getElementById("btnAI").addEventListener("click",function()
{
    ai.showInfo();
});

document.getElementById("btnEdit").addEventListener("click",function()
{
    editing.showInfo();
});

document.getElementById("btnGame").addEventListener("click",function()
{
    game.showInfo();
});
function changeMessage()
{
    let name = prompt("Enter your Name");

    document.getElementById("message").innerHTML=
    "Welcome "+name+"!";
}

function changeColor()
{
    const colors=
    [
        "#FFDAB9",
        "#90EE90",
        "#87CEEB",
        "#E6E6FA",
        "#F4A460"
    ];

    const randomColor=
    colors[Math.floor(Math.random()*colors.length)];

    document.body.style.backgroundColor=randomColor;
}

</script>

</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>element.addEventListener('click', func)</code>	Attaches an event handler to an element without overwriting existing event handlers.
<code>let name = prompt('Enter Name');</code>	Displays a dialog box that prompts the visitor for input.
<code>document.body.style.background = '...'</code>	Directly manipulates the CSS background property of the document body node.
<code>class AdvancedCourse extends Course</code>	Establishes a prototype chain where AdvancedCourse inherits from Course.
<code>document.getElementById(this.id).innerHTML</code>	Dynamically updates the HTML content of the specific element tied to the object instance.

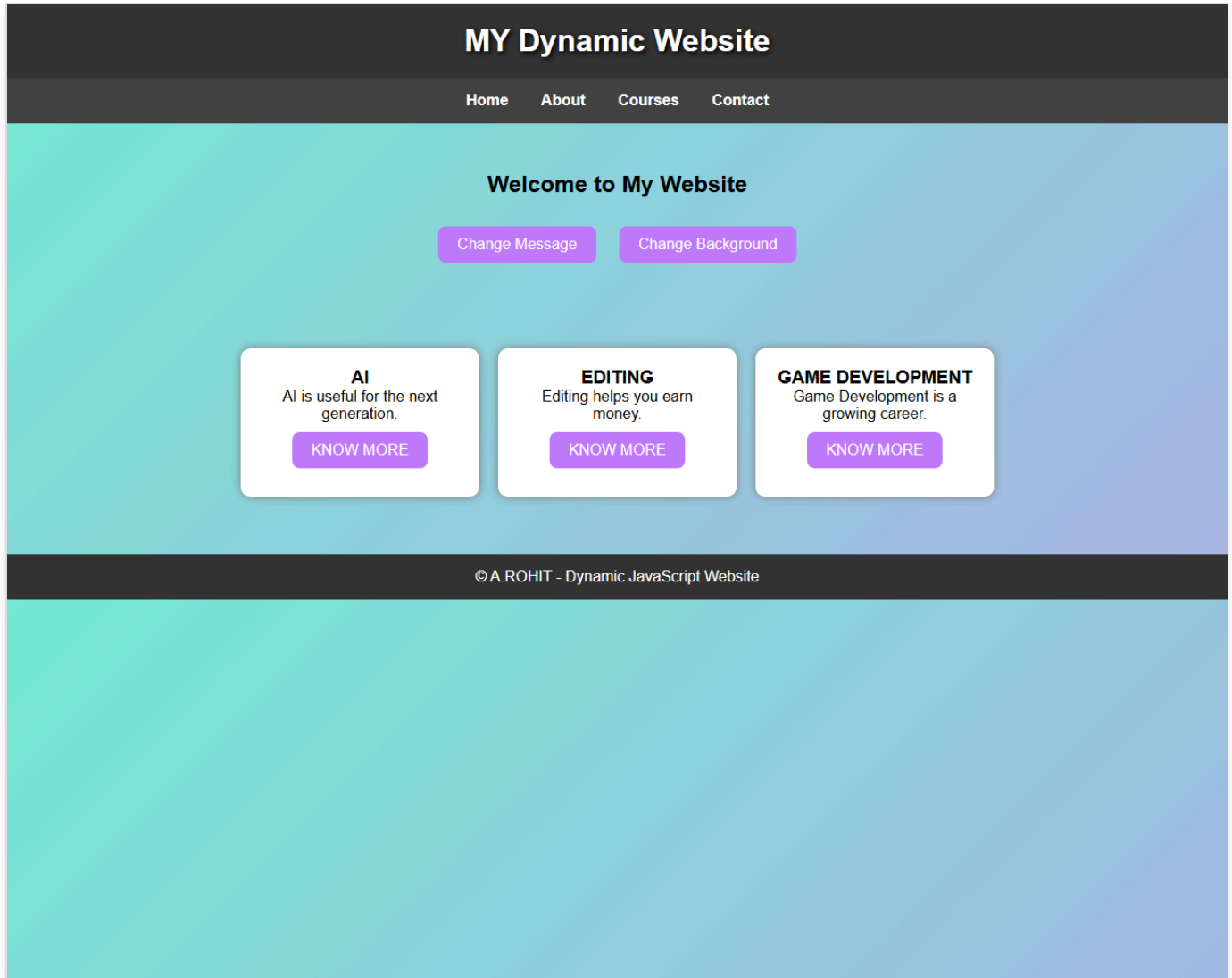
8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Class Instantiation & Method Call	Click 'Open' button on AI course card	AI object executes <code>showInfo()</code> injecting details into DOM	Pass
TC-02	Style Mutation Event	Click the 'Change Color' theme button	Body background gradient updates to linear-gradient dynamically	Pass
TC-03	Synchronous Prompt Event	Click 'Message' button and enter name 'Rohit'	DOM heading updates to 'Welcome Rohit' successfully	Pass
TC-04	Null Prompt Handling	Click 'Message' button and hit 'Cancel'	DOM heading updates to 'Welcome null' due to missing validation	Fail
TC-05	Unbound Event Listener	Click a button without an attached listener	No action occurs, DOM remains unchanged	Fail

9. OUTPUT

Output 1: Initial UI Render Before Interactions (Passed Output)



10. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.